# PPA Speech Therapy Suite

## Technical Reference Documentation

Version 4 (March 2026)

Single-file React application for Primary Progressive Aphasia (PPA) speech therapy. Runs as a Claude.ai artifact or locally via Vite. Powered by Claude Sonnet.

# 1. Architecture Overview

The PPA Speech Therapy Suite is a single-page React application that runs entirely in the browser. There is no backend server — all persistence is via localStorage and all AI calls are made directly from the browser to the Anthropic API.

## 1.1 Technology Stack

| Layer | Technology |
|---|---|
| UI framework | React 18 (hooks only, no class components) |
| Build tool | Vite 5 with ESM modules |
| AI / LLM | Anthropic Claude (claude-sonnet-4-5 or latest) |
| Styling | Inline styles only — no CSS framework |
| Speech synthesis | meSpeak v2 (eSpeak compiled to JavaScript via Emscripten) |
| Distribution | Claude.ai artifact (single JSX file) or Vite dev/build |
| Persistence | localStorage (all keys prefixed ppa_) |
| PWA | manifest.json + service worker for iPad home-screen install |

## 1.2 Source File Layout

```
ppa-source/
  ppa-speech-therapy_main.jsx   # App shell + all modules except Naming and SentenceBuilder
  NamingModule.jsx              # Picture-naming practice with spaced repetition
  SentenceBuilderModule.jsx     # Visual drag-and-drop sentence construction
  ExportImportSystem.jsx        # .ppa / .ppabak export, import, and backup logic
  shared.jsx                    # Shared utilities: fetchAnthropicApi, CallAPI, ThinkingDots, ZoomableGraphic
  data/
    config.js                   # CLAUDE_MODEL constant + Dr. Aria SYSTEM_PROMPT
    dictionary.js               # Unified word->{graphic, cues, categories} store
    namingItems.js              # 10 built-in picture-naming items (seed data)
    sbWordBank.js               # Noun/verb/adjective/adverb/pronoun/prep/article banks
    sbConjugation.js            # Verb conjugation rules for all tenses
    repetitionItems.js          # Repetition drill levels
    sentenceTasks.js            # Sentence completion and construction prompts
    scripts.js                  # Functional phrase scripts
    assessmentTasks.js          # Evaluation items
    videoClips.js               # Video comprehension clips and question types
    tools.js                    # Sidebar navigation definitions
```

## 1.3 Module Dependencies

All inter-module shared code lives in shared.jsx. Modules import only what they need. The dependency graph is intentionally flat — no circular imports.

## 2. Shared Utilities (shared.jsx)

All Anthropic API access and the loading indicator live in shared.jsx. Never duplicate these inline in a module.

### 2.1 fetchAnthropicApi(body, signal?)

Low-level async helper. Applies all required headers (x-api-key, anthropic-version, anthropic-dangerous-direct-browser-access). Returns parsed JSON. Throws on network error or abort. Use this for fire-and-forget calls (e.g. emoji lookup in SentenceBuilder).

```
const data = await fetchAnthropicApi({
  model: CLAUDE_MODEL,
  max_tokens: 256,
  messages: [{ role: 'user', content: prompt }]
}, abortSignal);
```

### 2.2

React component that fires one API request on mount and calls onResult(text) or onError(err) exactly once. Uses AbortController — the request is cancelled automatically on unmount. onResult always receives a non-empty string (falls back to "Well done — keep going!"). Mount it conditionally: {pendingAI && <CallAPI … />}.

### 2.3

Animated three-dot spinner. Use during any AI loading state.

### 2.4

Wraps any graphic (emoji or image) and makes it clickable. On click, a full-screen modal overlay opens showing the graphic at twice its normal display size, with a white background and border. The overlay closes when the user clicks outside the graphic or presses **Escape**. The ESC listener is registered only while the overlay is open and is cleaned up on close.

Props: graphic (string — emoji or image src), alt (string), width / height (px, for image elements), fontSize (px, for emoji spans), imgStyle / spanStyle (merged into the display element).

Used by: NamingModule (practice card, item list) and AssessmentModule (confrontation naming task). Import from shared.jsx; never inline.

# 3. Data Layer

## 3.1 localStorage Keys

| Key | Contents |
|---|---|
| ppa_dictionary | Canonical word→{graphic, cues, categories} map (JSON) |
| ppa_naming_items | Current naming practice list (JSON array) |
| ppa_naming_sr | Spaced repetition state per word (JSON object) |
| ppa_naming_audio_hints | Boolean — audio hints on/off (string 'true'/'false') |
| ppa_session_log | Array of session log entries (JSON) |
| ppa_progress | Accuracy history per module (JSON) |
| ppa_scripts | Custom scripts (JSON array) |
| ppa_sentences | Custom sentence tasks (JSON array) |
| ppa_video_clips | Custom video clips (JSON array) |
| ppa_assessment | Custom assessment tasks (JSON array) |

## 3.2 Dictionary (data/dictionary.js)

Single source of truth for all word graphics, stored in localStorage under ppa_dictionary. Both NamingModule and SentenceBuilder read and write through the dictionary API:

- dictLoadNamingItems() / dictSaveNamingItems(items) — load/persist the naming practice list.
- dictGetGraphic(word, fallback) — resolve canonical emoji or base64 image.
- dictAddWord(word, graphic) — register a new word (first writer wins; ■ is always upgradeable).
- useDictionaryLookup() — React hook returning a stable { word: graphic } map.

## 4. Export / Import System (ExportImportSystem.jsx)

### 4.1 File Formats

**.ppa files** — per-module item exports. Format: { ppaExport: true, moduleId, items: [...] }.

**.ppabak files** — full-app backup of all ppa_* localStorage keys. Restoring reloads the page.

### 4.2 Public API

All public functions and components are named exports. The main app and each module import only what they need.

| Export | Type | Purpose |
|---|---|---|
| ppaDownload(filename, data) | function | Triggers browser file download |
| ppaDoBackup() | function | Exports full .ppabak backup |
| ppaBackupIsStale() | function | Returns true if last backup >7 days ago |
| PpaAdminToolbar | component | Admin mode toolbar strip |
| PpaExportDialog | component | Export dialog for a module |
| PpaReexportDialog | component | Re-export changed items dialog |
| BackupRestorePanel | component | Full backup/restore UI panel |

# 5. Naming Module (NamingModule.jsx)

## 5.1 Practice Phases

The naming flow moves through these phases for each item:

| Phase | Display | Transitions |
|---|---|---|
| show | Graphic + empty input | Space → phoneme hint; Semantic Cue btn → semantic; type + Got it! → record correct |
| semantic | Semantic cue text shown | Phonemic Cue btn → phonemic |
| phonemic | Phonemic cue shown | Reveal btn → answer |
| answer | Full word shown | Next word → advance |

## 5.2 Spaced Repetition Engine

PPA-adapted SR — deliberately conservative (max 5-day interval, regression decay on every load):

| Result | SR_FACTOR | When used |
|---|---|---|
| correct | ×1.2 | Answered without any help |
| space_cued | ×0.9 | Used the Space-key phoneme starter |
| semantic_cued | ×0.8 | Used the concept hint |
| phonemic_cued | ×0.5 | Used the sound cue |
| failed | reset → 1 day | Needed full reveal |

SR state is stored in localStorage under ppa_naming_sr: { [word]: { interval, dueDate, streak, lastResult, lastSeen } }.

Words answered with phonemic_cued or failed are re-inserted 4 positions ahead in the session queue for same-session repetition.

## 5.3 AI Feedback Flow

After recording any response, <CallAPI> is mounted to fetch Dr. Aria's feedback. Once the response arrives the "Next word →" button appears. If the API call fails (no key, CORS, etc.) onError calls next() directly so the user is never left stuck.

### 5.4 Phoneme Starter (Space Key) with Audio Hints

In the show phase, when the response input is empty, pressing **Space** reveals a prefix of the target word and (if audio hints are on) speaks it aloud via **meSpeak**.

#### Audio-hints toggle

A ■ button in the bottom-right corner of the input area toggles audio hints on/off. The choice is persisted in localStorage under ppa_naming_audio_hints. Default is on.

#### Audio-on mode (default)

Each Space press advances the display to the next vowel boundary (via findNextVowelEnd) plus the immediately following consonant group (via getNextPhoneme as an *anchor*), then speaks that exact text:

```
const newCharCount = findNextVowelEnd(item.word, phonemesRevealed);
const anchor = newCharCount < item.word.length
  ? getNextPhoneme(item.word, newCharCount)   // next consonant group
  : "";
const displayCount = newCharCount + anchor.length;
setPhonemesRevealed(displayCount);
meSpeak.resetQueue();
meSpeak.speak(item.word.slice(0, displayCount).toLowerCase(), { speed: 130 });
```

The trailing consonant anchor is essential for correct vowel quality. Without it, eSpeak's LTS rules assign the wrong allophone to word-final vowels: meSpeak.speak("gla") produces a schwa-like sound. With the anchor, meSpeak.speak("glas") forces a CVC (closed syllable) context and correctly produces the short /æ/ of *glasses*. Similarly, speaking "chair" gives eSpeak's dictionary pronunciation /t■■r/ rather than the standalone-word pronunciation /t■a■/ of "chai".

Display and audio always show the same characters — the patient never hears more than is displayed.

| Example | Display | speakText | Result |
|---|---|---|---|
| glasses, press 1 | GLAS | "glas" | short-/æ/ ✓ |
| glasses, press 2 | GLASSES | "glasses" | full word ✓ |
| chair, press 1 | CHAIR | "chair" | /t■■r/ not /t■a■/ ✓ |
| apple, press 1 | AP | "ap" | short-/æ/ ✓ |

#### Audio-off mode

Each Space press advances by one grapheme group (1–3 characters) using getNextPhoneme. No audio is played.

#### Vowel and grapheme helpers

| Function | Purpose |
|---|---|
| findNextVowelEnd(word, startPos) | Returns char position after next vowel group (single or digraph). |
| getNextPhoneme(word, pos) | Returns next grapheme cluster (1–3 chars) via TRIGRAPHS / DIGRAPHS arrays; used for both audio-off advances and the audio-on anchor. |
| VOWEL_GROUPS | Multi-letter vowel digraphs (ai, ay, ea, ee, igh, ough, …) checked longest-first. |

| Function | Purpose |
|---|---|
| LETTER_VOWELS | Set: {a, e, i, o, u}. |

### meSpeak initialisation

```
import meSpeak from "mespeak";
import meSpeakConfig from "mespeak/src/mespeak_config.json";
import enVoice from "mespeak/voices/en/en-us.json";

if (!meSpeak.isConfigLoaded()) meSpeak.loadConfig(meSpeakConfig);
if (!meSpeak.isVoiceLoaded())  meSpeak.loadVoice(enVoice);
```

Initialisation runs at module level. meSpeak.resetQueue() is called before each speak() to prevent audio queuing when Space is pressed rapidly.

### SR result

Submitting with phonemesRevealed > 0 records space_cued (SR_FACTOR ×0.9). The mild penalty acknowledges the word was not recalled unaided, while reflecting that audio support is lighter than semantic or phonemic cueing.

### 5.5 AI-Assisted Item Defaults

When the **Word** field in the Add Item form (AdminPanel → ItemForm) loses focus and contains a non-empty value, generateDefaults(word) is called via fetchAnthropicApi. The function requests a JSON object from the Claude API containing four fields:

- graphic — a single emoji that best represents the word
- category — a short category label (e.g. “fruit”)
- clue_semantic — a short sentence cue (e.g. “It's a fruit you eat”)
- clue_phonemic — a phonemic cue noting the starting sound (e.g. “Starts with 'A'…”)

Each field is applied only if it is still at its blank default (■ for graphic, empty string for the rest) — any value already typed by the admin is preserved. A <ThinkingDots /> spinner is shown during the request. Failures are silent; all fields remain editable. Requires an internet connection and a valid API key.

### 5.6 BulkImportPanel

Renders inside AdminPanel when mode === "bulkimport". Lets the admin select multiple image files at once (drag-and-drop or multi-select file browser; accepts image/*, .heic, .heif). Each file is compressed to a base64 JPEG via compressImageFile (240 px max side, 0.75 quality) and added as a draft row in an editable grid table.

#### Grid columns

- **Photo** — compressed thumbnail (52 × 52 px).
- **Word** — text input; blurring the field fires generateForDraft(id, word), which calls fetchAnthropicApi to suggest category, clue_semantic, and clue_phonemic. Only empty fields are filled. A ThinkingDots spinner appears during the request.
- **Category, Semantic cue, Phonemic cue** — free-text inputs.
- **Remove** — deletes the row from the draft list.

"Generate AI defaults for all" iterates the draft list sequentially, calling generateForDraft for each row that has a non-empty word.

"Save N items" calls onSaveAll passing rows where all four fields (word, category, clue_semantic, clue_phonemic) are non-empty. handleBulkSave deduplicates against the existing item list by word (case-insensitive) before appending.

Filename inference: camera-roll names matching /^(img|dsc|photo|picture|p|image|pic)[-_]?\d+$/i are left blank; other filenames are lowercased with hyphens and underscores converted to spaces and used as the initial word value.

### 5.7 Generate by Category

Renders inside AdminPanel when mode === "generate". Lets the admin auto-populate a category with the most common nouns using a single AI call.

#### Controls

- **Category input** — free-text field for the category name (e.g. "fruit", "animal", "clothing").
- **Count** — number input, 1–50, default 15.
- **Generate** — fires a single fetchAnthropicApi call requesting the N most common nouns in the category. The prompt asks for a JSON array of objects, each with word, graphic (emoji), clue_semantic, and clue_phonemic. A <ThinkingDots /> spinner is shown while the request is in flight.

### Results grid

Uses the same grid style as BulkImportPanel. Columns: checkbox, Graphic, Word, Category, Semantic cue, Phonemic cue, Status.

Each generated noun is classified via checkDuplicate():

- **New items** — shown with a green **NEW** badge and pre-selected checkbox. Category is set to the new category name.
- **Duplicates** — shown with an amber **EXISTS** badge, no checkbox, and all existing values including the original category displayed in amber. Duplicates are never added.

A summary bar above the grid shows the count of new and existing items, plus a "Select all new" checkbox. The grid scrolls independently; controls and the "Add N items to library" action button remain pinned.

### Actions

"Add N items to library" calls handleAddGenerated(), which filters to selected new items, strips the internal _status field, assigns a custom-{timestamp} id to each, appends them to the item list via onUpdate, and returns to list view.

If the API call fails (no key, network error), an alert is shown and the controls remain active so the admin can retry.

## 6. Sentence Builder Module (SentenceBuilderModule.jsx)

### 6.1 Word Bank

Words are organised into seven categories: nouns, verbs, adjectives, adverbs, pronouns, prepositions, and articles. The bank is stored in data/sbWordBank.js. Conjugation rules for all tenses are in data/sbConjugation.js.

### 6.2 Drag-and-Drop

Built with native HTML5 drag-and-drop events (no third-party library). Touch events on iOS are handled with a polyfill embedded in the component. Tiles can be dragged from the bank into the tray, and reordered within the tray.

### 6.3 AI Feedback

When the patient taps **Check sentence**, the constructed sentence is sent to fetchAnthropicApi with a prompt asking Dr. Aria to give brief, encouraging grammatical feedback. The emoji for new words is also looked up via fetchAnthropicApi and stored in the dictionary.

# 7. Other Modules

## 7.1 AI Therapist (TherapistModule)

Full multi-turn conversation with Dr. Aria. Messages are accumulated in component state (not persisted). Uses <CallAPI> mounted on every send. The system prompt is defined in data/config.js.

## 7.2 Assessment

Presents tasks from data/assessmentTasks.js. Responses are recorded as correct/incorrect/partial and logged to the session log. Admin can add, edit, and reorder tasks.

## 7.3 Repetition

Drills from data/repetitionItems.js, grouped by level. Accuracy is tracked per level. Dr. Aria gives end-of-level feedback.

## 7.4 Script Training

Scripts from data/scripts.js. Each script has a name, context, and ordered lines. After completing a script, Dr. Aria gives feedback. Scripts can be exported/imported as .ppa files.

## 7.5 Sentence Work

Two sub-modes (completion and construction) drawing from data/sentenceTasks.js. Both use <CallAPI> for per-item AI feedback.

## 7.6 Video Questions

Clips defined in data/videoClips.js. YouTube embeds via iframe. Questions can be multiple-choice (EMOJI_OPTIONS) or open-ended. Responses are logged.

## 8. Progress Module

Reads from the session log (ppa_session_log) and the SR state (ppa_naming_sr) to display:

- **Session log** — timestamped list of all activity this session.
- **Accuracy charts** — per-module correct/incorrect percentages.
- **SR table** — per-word interval, streak, last result, next due date.

The session log is not persisted across page reloads — it is held in React state at the app root and passed down as a prop.

## 9. Configuration (data/config.js)

config.js exports two constants:

**CLAUDE_MODEL** — the Anthropic model string used in all API calls (e.g. "claude-sonnet-4-5"). Change this to upgrade to a newer model.

**SYSTEM_PROMPT** — Dr. Aria's full system prompt. Defines her persona, tone, and therapeutic approach. Edit this to customise her behaviour.

### 9.1 Environment Variable

The Anthropic API key is read from import.meta.env.VITE_ANTHROPIC_API_KEY. Create a .env file in the repo root:

```
VITE_ANTHROPIC_API_KEY=sk-ant-...
```

Without it the AI calls fail silently and the app auto-advances rather than showing Dr. Aria's feedback.

# 10. Implementation Notes

### 10.1 React StrictMode

StrictMode is active in development (src/main.jsx). Effects run twice. All useEffect hooks must return a cleanup function. <CallAPI>'s AbortController handles this automatically.

### 10.2 No Backend — Security Considerations

The API key is stored in browser localStorage (or .env for Vite builds). This is acceptable for a single-patient, single-device use case but is not suitable for a multi-user web deployment. In that scenario the key must be moved to a backend proxy.

### 10.3 CORS — anthropic-dangerous-direct-browser-access

Anthropic's API requires the header anthropic-dangerous-direct-browser-access: true for direct browser calls. This is set in fetchAnthropicApi and must not be removed.

### 10.4 Admin PIN

The PIN is defined as a constant in NamingModule.jsx (const ADMIN_PIN = "1234"). Change this before clinical deployment. The PIN is checked only in the browser — it provides no server-side security.

### 10.5 PWA / Service Worker

The pwa/ directory contains a manifest.json and a service-worker.js that cache the app shell for offline use. The service worker is registered in index.html. AI features remain unavailable offline.

### 10.6 localStorage Limits

Most browsers allow 5–10 MB of localStorage per origin. Base64-encoded photos in the dictionary are the largest consumers. The app does not enforce a limit — if storage fills, writes will silently fail. Advise users to use emoji graphics rather than photos where possible.

### 10.7 Bundle Size

The Vite build produces a single JS bundle. The meSpeak library (eSpeak WASM) adds ~1.5 MB to the bundle. This is acceptable for a local/LAN deployment but may be slow on a first load over a slow connection. Consider using import() lazy loading for meSpeak if load time is a concern.

### 10.8 Audio Hints — Vowel Anchoring Approach

The initial approach used phonemizer (Xenova, eSpeak-NG WASM) to convert each word to IPA, map IPA tokens to eSpeak X-SAMPA notation, and call meSpeak.speak("[[g l {]]") in phoneme mode. Two problems made this unworkable: (1) the phonemizer package failed to initialise its search engine in the Vite development environment; (2) meSpeak v2's [[...]] phoneme notation produced no audio output (returns null for rawdata: true).

The replacement approach requires no external phonemizer. Speaking the prefix plus one anchor consonant in text mode (meSpeak.speak("glas")) achieves correct vowel quality because eSpeak's LTS rules handle closed-syllable /æ/ correctly. For dictionary words the full matched string invokes eSpeak's internal dictionary, giving the correct pronunciation automatically.

phonemizer and espeak-ng have been removed from package.json. Both are listed in vite.config.js → optimizeDeps.exclude to prevent Vite from attempting to pre-bundle any stale package stubs.

# 11. Installer / Distribution

## 11.1 macOS Installer (installer/install.sh)

A shell script that:

- Checks for Node.js and npm.
- Runs npm install.
- Copies PDF documentation to the install directory.
- Performs a preflight check for PDF presence.
- Launches the Vite dev server.

## 11.2 Windows Installer (win-installer/install.ps1)

A PowerShell script performing the same steps as the macOS installer, adapted for Windows paths and conventions.

## 11.3 Claude.ai Artifact

The bundle files (ppa-speech-therapy-bundle.jsx) in each installer directory are single-file versions of the app suitable for pasting into a Claude.ai artifact. They include all module code inlined.

## 12. Changelog

### Version 4.0.1 (March 2026)

- Added click-to-zoom for graphics: tapping any picture card in Naming Practice or Assessment opens a 2× popup. Press Escape or click outside to close.
- Added ZoomableGraphic shared component to shared.jsx.

### Version 4.0.0 (March 2026)

- Added meSpeak audio hints with vowel-anchored pronunciation in Naming module.
- Replaced phonemizer / espeak-ng IPA pipeline with text-mode meSpeak.
- Added ppa_naming_audio_hints localStorage key for toggle persistence.
- Added Space-bar phoneme starter (space_cued, SR ×0.9).
- Extracted shared utilities to shared.jsx (fetchAnthropicApi, CallAPI, ThinkingDots).
- Renamed main source file to ppa-speech-therapy_main.jsx.
- Added Vite project scaffold (package.json, vite.config.js, index.html, src/main.jsx).
- Added iPad PWA (manifest.json + service worker).
- PDF documentation bundled into macOS and Windows installers.
- Max SR interval corrected to 5 days.